

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT
TP. HỒ CHÍ MINH
Khoa Điện - Điện tử
Bộ môn Điện tử - Thông tin

Thực hành Học máy và Trí tuệ nhân tạo

Bài 5: Object Detection with YOLOv11

Giảng viên: TS. Huỳnh Thế Thiện & PGS. TS Trương Ngọc Sơn

Tháng 03, 2026

1 Mục tiêu

Sau khi hoàn thành bài thực hành này, sinh viên có thể:

- Hiểu bài toán Object Detection
- Hiểu nguyên lý hoạt động của YOLO
- Huấn luyện mô hình YOLOv11
- Đánh giá mô hình bằng mAP
- Phân tích lỗi của mô hình
- Cải thiện mô hình detection

2 Object Detection

Trong bài toán **Image Classification**:

$$1 \text{ image} \rightarrow 1 \text{ label}$$

Ví dụ:

- cat
- dog

- airplane

Tuy nhiên trong thực tế một ảnh có thể chứa nhiều đối tượng.

Object Detection cần dự đoán:

- class label
- vị trí object (bounding box)

Bounding box được biểu diễn bằng:

- x center
- y center
- width
- height

3 YOLO Algorithm

YOLO (You Only Look Once) là một thuật toán Object Detection nổi tiếng.

Đặc điểm:

- tốc độ nhanh
- real-time detection
- độ chính xác cao

YOLO chia ảnh thành nhiều grid và dự đoán:

- bounding boxes
- class probabilities

Trong bài này chúng ta sử dụng:

YOLOv11

4 Chuẩn bị môi trường

Chạy trên Kaggle GPU.

```
!pip install ultralytics
```

Import thư viện:

```
from ultralytics import YOLO
```

```
import matplotlib.pyplot as plt
import cv2
import os
import torch
```

Kiểm tra GPU:

```
print(torch.cuda.is_available())
```

5 Dataset

Dataset sử dụng:

Self Driving Cars Dataset (Kaggle)

Dataset gồm:

- ảnh đường phố
- bounding boxes
- classes: car, pedestrian, traffic light

Cấu trúc dataset YOLO:

```
dataset/
  images/
    train/
    val/

  labels/
    train/
    val/
```

6 YOLO Annotation Format

Ví dụ một file label:

```
0 0.52 0.48 0.22 0.30
```

Trong đó:

- 0 : class id
- 0.52 : x center

- 0.48 : y center
- 0.22 : width
- 0.30 : height

Tất cả giá trị đều được **normalize** từ 0 đến 1.

7 Visualization Bounding Boxes

Code hiển thị bounding box từ file label.

```
import matplotlib.pyplot as plt

image_path = "image.jpg"
label_path = "image.txt"

img = cv2.imread(image_path)
h, w, _ = img.shape

with open(label_path) as f:
    lines = f.readlines()

for line in lines:

    cls, xc, yc, bw, bh = map(float, line.split())

    x1 = int((xc - bw/2) * w)
    y1 = int((yc - bh/2) * h)
    x2 = int((xc + bw/2) * w)
    y2 = int((yc + bh/2) * h)

    cv2.rectangle(img, (x1, y1), (x2, y2), (0, 255, 0), 2)

plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.show()
```

8 Visualization Ground Truth Bounding Box

Trước khi huấn luyện mô hình, cần kiểm tra các annotation trong dataset.

Mỗi ảnh trong YOLO có một file label tương ứng:

```
image_001.jpg
image_001.txt
```

File label chứa nhiều dòng, mỗi dòng là một object.

Ví dụ:

```
0 0.52 0.48 0.22 0.30
1 0.31 0.60 0.10 0.15
```

Trong đó:

- số đầu tiên: class id
- các số còn lại: bounding box

Sinh viên cần kiểm tra:

- bounding boxes có đúng vị trí không
- annotation có lỗi không

Code đọc label và vẽ bounding box:

```
import cv2
import matplotlib.pyplot as plt

image_path = "/kaggle/input/self-driving-cars/images/
train/0001.jpg"
label_path = "/kaggle/input/self-driving-cars/labels/
train/0001.txt"

img = cv2.imread(image_path)
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

h, w, _ = img.shape

with open(label_path) as f:
    labels = f.readlines()

for line in labels:

    cls, xc, yc, bw, bh = map(float, line.split())

    x1 = int((xc - bw/2) * w)
    y1 = int((yc - bh/2) * h)
```

```

x2 = int((xc + bw/2) * w)
y2 = int((yc + bh/2) * h)

cv2.rectangle(img, (x1, y1), (x2, y2), (255, 0, 0), 2)

plt.figure(figsize=(6, 6))
plt.imshow(img)
plt.axis("off")

```

Sinh viên nên thử với nhiều ảnh khác nhau trong dataset.

9 Load YOLOv11 Model

```
model = YOLO("yolo11n.pt")
```

10 Code mẫu hoàn chỉnh

Phần này cung cấp một pipeline hoàn chỉnh để huấn luyện và đánh giá mô hình YOLOv11. Sinh viên cần chạy toàn bộ code trước khi thực hiện các bài tập.

10.1 Cài đặt thư viện

```
!pip install ultralytics -q
```

10.2 Import thư viện

```

from ultralytics import YOLO
import matplotlib.pyplot as plt
import cv2
import torch
import os

```

10.3 Kiểm tra GPU

```
print("GPU available:", torch.cuda.is_available())
```

Khi chạy trên Kaggle cần bật:

- GPU
- Accelerator: Tesla T4

10.4 Load mô hình YOLOv11

```
model = YOLO("yolo11n.pt")
```

Mô hình yolo11n là phiên bản nhỏ, phù hợp để huấn luyện nhanh trong bài thực hành.

10.5 Kiểm tra dataset

```
dataset_path = "/kaggle/input/self-driving-cars"

print(os.listdir(dataset_path))
```

Sinh viên cần kiểm tra dataset có các thư mục:

- images
- labels
- data.yaml

10.6 Hiển thị một ảnh trong dataset

```
image_path = "/kaggle/input/self-driving-cars/images/
train/0001.jpg"

img = cv2.imread(image_path)
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

plt.figure(figsize=(6,6))
plt.imshow(img)
plt.axis("off")
```

10.7 Huấn luyện mô hình

```
results = model.train(

    data="/kaggle/input/self-driving-cars/data.yaml",

    epochs=20,
```

```
        imgsz=640,

        batch=16,

        device=0
    )
```

Trong quá trình huấn luyện, mô hình sẽ hiển thị:

- box loss
- classification loss
- precision
- recall
- mAP

10.8 Đánh giá mô hình

```
metrics = model.val()

print(metrics)
```

Sinh viên cần quan sát các chỉ số:

- Precision
- Recall
- mAP50

10.9 Dự đoán trên tập test

```
results = model.predict(

    source="/kaggle/input/self-driving-cars/test/images",

    conf=0.25,

    save=True
)
```

Kết quả sẽ được lưu vào thư mục:

```
/kaggle/working/runs/detect/predict
```


10.10 Hiển thị kết quả detection

```
result_image = "/kaggle/working/runs/detect/predict/
image0.jpg"

img = cv2.imread(result_image)
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

plt.figure(figsize=(8,8))
plt.imshow(img)
plt.axis("off")
```

Sinh viên sẽ thấy:

- bounding boxes
- class labels
- confidence score

10.11 Xuất mô hình

```
model.export(format="onnx")
```

11 Huấn luyện mô hình

```
model.train(
    data="data.yaml",
    epochs=20,
    imgsz=640,
    batch=16,
    device=0
)
```

12 Đánh giá mô hình

```
metrics = model.val()
print(metrics)
```

Các chỉ số quan trọng:

- Precision
- Recall
- mAP

13 Intersection over Union (IoU)

Một chỉ số quan trọng trong Object Detection là:

Intersection over Union (IoU)

IoU đo mức độ chồng lấp giữa:

- Ground Truth Bounding Box
- Predicted Bounding Box

Công thức:

$$IoU = \frac{Area\ of\ Intersection}{Area\ of\ Union}$$

Giá trị IoU nằm trong khoảng:

$$0 \leq IoU \leq 1$$

- IoU gần 1: dự đoán rất chính xác
- IoU gần 0: dự đoán sai

Ví dụ tính IoU bằng Python:

```
def compute_iou(boxA, boxB):  
  
    xA = max(boxA[0], boxB[0])  
    yA = max(boxA[1], boxB[1])  
    xB = min(boxA[2], boxB[2])  
    yB = min(boxA[3], boxB[3])  
  
    interArea = max(0, xB-xA) * max(0, yB-yA)  
  
    boxAArea = (boxA[2]-boxA[0])*(boxA[3]-boxA[1])  
    boxBArea = (boxB[2]-boxB[0])*(boxB[3]-boxB[1])  
  
    iou = interArea / float(boxAArea + boxBArea -  
        interArea)  
  
    return iou
```

```
box1 = [100,100,300,300]
box2 = [150,150,350,350]

print(compute_iou(box1,box2))
```

14 Prediction

```
results = model.predict(
    source="test_images/",
    conf=0.25,
    save=True
)
```

15 Training Results

YOLO tự động lưu kết quả training:

```
/runs/detect/train/
```

Bao gồm:

- training curves
- confusion matrix
- predictions

Sinh viên cần quan sát các biểu đồ:

- box loss
- classification loss
- mAP

16 Visualization Training Loss

Sau khi huấn luyện YOLO, thư mục:

```
/runs/detect/train/
```

chứa file:

```
results.csv
```

File này chứa các chỉ số training theo từng epoch.

Sinh viên có thể vẽ biểu đồ training loss.

```
import pandas as pd
import matplotlib.pyplot as plt

results = pd.read_csv(
    "/kaggle/working/runs/detect/train/results.csv"
)

plt.plot(results['train/box_loss'])
plt.plot(results['val/box_loss'])

plt.xlabel("epoch")
plt.ylabel("loss")

plt.legend(["train", "validation"])

plt.title("Training Loss Curve")

plt.show()
```

Sinh viên cần quan sát:

- loss có giảm theo thời gian hay không
- validation loss có cao hơn train loss không

Nếu validation loss tăng mạnh có thể xảy ra:

- overfitting

17 Error Analysis

Sau khi huấn luyện mô hình, cần phân tích các lỗi của mô hình.

Các lỗi phổ biến trong Object Detection:

- False Positive
Mô hình dự đoán có object nhưng thực tế không có.
- False Negative
Mô hình không phát hiện object trong ảnh.

- Localization Error

Mô hình phát hiện object nhưng bounding box không chính xác.

Sinh viên cần mở thư mục:

```
/kaggle/working/runs/detect/predict
```

và quan sát các ảnh kết quả.

Câu hỏi cần trả lời:

- Những object nào khó detect nhất?
- Các bounding box có chính xác không?
- Những trường hợp nào mô hình dự đoán sai?
- Nguyên nhân có thể do đâu?

18 Bài tập

18.1 Bài tập 1: Dataset Exploration

Thực hiện:

- đếm số ảnh train
- đếm số ảnh validation
- liệt kê các classes

18.2 Bài tập 2: Visualization

Hiển thị 5 ảnh từ dataset và vẽ bounding boxes.

18.3 Bài tập 3: Huấn luyện YOLO

Huấn luyện mô hình với:

- epochs = 10
- epochs = 20
- epochs = 40

So sánh mAP.

18.4 Bài tập 4: Model Comparison

Thử các model:

- yolo11n
- yolo11s
- yolo11m

So sánh:

- training time
- mAP

18.5 Bài tập 5: Image Size

Thử:

- 320
- 640

- 800

Quan sát ảnh hưởng đến accuracy.

18.6 Bài tập 6: Data Augmentation

Thêm augmentation:

- flip
- rotate
- brightness

Quan sát sự thay đổi của mAP.

18.7 Bài tập 7: Prediction

Chạy mô hình trên 10 ảnh test và lưu kết quả.

18.8 Bài tập 8: Video Detection

Chạy mô hình trên video.

Hiển thị bounding boxes.

18.9 Bài tập 9: Error Analysis

Tìm các trường hợp:

- False Positive
- False Negative

Giải thích nguyên nhân.

18.10 Bài tập 10: Mini Challenge

Sinh viên cần cải thiện mô hình để đạt:

$$\text{mAP} > 0.65$$

Các phương pháp có thể thử:

- tăng epochs
- tăng image size
- thử model lớn hơn

- thêm data augmentation